

Segurança da Informação & Consultoria Web3

Relatório de Auditoria de Segurança

IModularCompliance (Interface)

Cliente	AxiaDigitalSolutions
Preparado por	HeimdallSec
Data	23/07/2026
Versão do Documento	2.0
Status da Auditoria	Final (correções aplicadas; itens informativos pendentes)

Commit auditado (Git): `9cd96c0b74c0c384d7e4daa934da7e5771494d7d`
https://github.com/AxiaDigitalSolutions/SmartContractsAudit/commit/9cd96c0b74c0c384d7e4daa934da7e5771494d7d

DOCUMENTO CONFIDENCIAL

Sumário

Sumário 2
Histórico de Versões 3
1. Resumo Executivo 3
1.1 Visão Geral do Escopo 3
1.2 Resumo das Descobertas (Revisão 2.0) 3
2. Metodologia de Auditoria 4
3. Descobertas Detalhadas (Findings) 4
[HSEC-MC-01] Ausência de Mecanismo de Confiança na Interface — Implementação Maliciosa pode Falsear Compliance 4
[HSEC-MC-02] Funções `created()` e `destroyed()` não distinguem chamador — Risco de Dupla Contagem 6
[HSEC-MC-03] Documentação NatSpec Incompleta — Comportamento de `canTransfer` vs `transferred` não especificado 7
4. Plano de Ação Priorizado (Novo na v2.0) 8
5. Recomendações Gerais e Melhores Práticas (Atualizado v2.0) 8
6. Conclusão da Revisão 2.0 8
7. Termo de Isenção de Responsabilidade (Disclaimer) 9

Histórico de Versões

Versão	Data	Autor	Descrição
1.0	18/06/2026	HeimdallSec	Auditoria inicial da interface IModularCompliance
2.0	23/07/2026	HeimdallSec	**Segunda revisão — reanálise após correções aplicadas pela equipe Axia; 2 findings Medium resolvidos e verificados; 1 finding Informational / Gas permanece em aberto (pendente)**

1. Resumo Executivo

A HeimdallSec realizou a **segunda revisão** da interface **IModularCompliance** para a **AxiaDigitalSolutions**. Esta reanálise confirma que a equipe Axia aplicou as correções recomendadas na versão 1.0: a interface foi atualizada e as mitigações associadas aos findings foram implementadas e verificadas.

Dos **3 achados** reportados originalmente (2 Medium, 1 Informational), os **2 findings Medium foram resolvidos** e verificados nesta revisão, enquanto o **1 finding Informational / Gas permanece em aberto** (correção ainda não aplicada pela equipe) — 2 de 3 resolvidas. Esta versão do relatório documenta as correções aplicadas para os findings Medium à luz das mudanças no ecossistema (`AxiaCompliantERC721` e implementações concretas), confirma a verificação de cada mitigação e consolida as melhores práticas adotadas.

1.1 Visão Geral do Escopo

Propriedade	Detalhe
Nome do Artefato	IModularCompliance
Cliente	AxiaDigitalSolutions
Tipo	Interface Solidity
Linguagem / Ecossistema	Solidity ^0.8.20 / EVM
Arquivos no Escopo	`interfaces/IModularCompliance.sol`
Linhas de Código (SLOC)	4 funções declaradas
Dependências	Nenhuma (interface pura)
Padrão de Referência	ERC-3643 (T-REX) — Modular Compliance

1.2 Resumo das Descobertas (Revisão 2.0)

Dos **3** problemas identificados na auditoria inicial, **2 foram resolvidos** (Medium) e **1 permanece em aberto** (Informational / Gas) — 2 de 3 resolvidas:

Severidade	Quantidade	Status de Resolução	Não Resolvidos	Parcialmente
Critical	0	**0 Resolvidos**	0	0
High	0	**0 Resolvidos**	0	0

Medium	2	2 Resolvidos	0	0
Low	0	0 Resolvidos	0	0
Informational / Gas	1	0 Resolvidos	1	0

> ***Total: 2 de 3 descobertas resolvidas. 1 finding Informational / Gas permanece em aberto (HSEC-MC-03).***

2. Metodologia de Auditoria

A metodologia aplicada nesta segunda revisão é idêntica à da versão 1.0, agora aplicada sobre o artefato já corrigido:

1. **Revisão de Compatibilidade:** Verificação das assinaturas contra a especificação T-REX/ERC-3643 e contra o consumo no `AxiaCompliantERC721`.
2. **Análise de Integração:** Identificação de riscos decorrentes do contrato entre a interface e sua implementação concreta.
3. **Modelagem de Ameaças:** Análise de cenários onde a implementação subjacente se comporta de forma maliciosa ou inesperada.

Adicional para v2.0: Foi realizada uma verificação de delta entre a versão anterior e a atual do arquivo `IModularCompliance.sol` para confirmar que as correções recomendadas foram efetivamente aplicadas e que cada finding foi resolvido.

3. Descobertas Detalhadas (Findings)

[HSEC-MC-01] Ausência de Mecanismo de Confiança na Interface — Implementação Maliciosa pode Falsear Compliance

Severidade: Medium **Status: Resolved Localização:** `interfaces/IModularCompliance.sol`, Linhas `1-22`

Descrição (original): A interface `IModularCompliance` define 4 funções (`canTransfer`, `transferred`, `created`, `destroyed`) sem qualquer mecanismo que permita ao contrato consumidor (`AxiaCompliantERC721`) verificar a integridade ou confiabilidade da implementação subjacente. O admin (`DEFAULT_ADMIN_ROLE`) pode trocar a implementação de compliance a qualquer momento via `setCompliance()`.

Uma implementação maliciosa poderia:

- `canTransfer()` sempre retornar `true`, ignorando todas as regras de compliance.
- `transferred()`/`created()`/`destroyed()` serem funções vazias (no-op), não registrando movimentações nos módulos.
- `canTransfer()` realizar chamadas externas que drenam gas ou causam DoS.

Reanálise v2.0:

A análise v2.0 confirmou que a equipe Axia aplicou as proteções recomendadas na interface e no consumidor. As considerações a seguir fundamentaram as mitigações implementadas:

1. **Ameaça de Upgrade Malicioso com Flash Loan:** Em cenários onde o `DEFAULT_ADMIN_ROLE` é um contrato multisig ou timelock, o risco é baixo. Porém, se o admin for uma EOA (prática desaconselhada), um comprometimento de chave privada permite que o atacante substitua a implementação por uma versão `canTransfer() = true` em uma única transação, sem qualquer período de observação.
2. **Ameaça de Reentrância via canTransfer():** Embora `canTransfer()` seja declarada como `view`, a Solidity não impõe a restrição `view` em chamadas via interface se a implementação concreta não respeitar a pureza. Uma implementação maliciosa poderia realizar chamadas externas dentro de `canTransfer()`, abrindo vetor de reentrância no `AxiaCompliantERC721`.
3. **Validação de Codehash:** Uma mitigação adicional não discutida na v1 seria armazenar o `codehash` da implementação aprovada e verificá-lo antes de cada chamada de compliance. Isto

impediria que uma implementação fosse substituída por SELFDESTRUCT seguido de CREATE2 no mesmo endereço.

Recomendação de Mitigação (refinada v2.0):

1. **Adicionar versionamento à interface:**

```
interface IModularCompliance {
    function VERSION() external pure returns (string memory);  // Ex: "1.0.0"
    function canTransfer(address from, address to, uint256 amount) external view returns (bool);
    // ...
}
```

1. **No AxiaCompliantERC721**, verificar a versão antes de aceitar uma nova implementação:

```
function setCompliance(address compliance_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (compliance_ == address(0)) revert ZeroAddress();
    require(
        keccak256(bytes(IModularCompliance(compliance_).VERSION())) ==
        keccak256(bytes("1.0.0")),
        "Unsupported compliance version"
    );
    compliance = IModularCompliance(compliance_);
    emit ComplianceSet(compliance_);
}
```

1. **Considerar upgradeable pattern com timelock** para trocas de compliance (ver HSEC-03 do relatório AxiaCompliantERC721).
2. **[NOVO v2.0] Verificação de codehash na implantação:** Adicionar uma verificação do `codehash` da implementação no momento do `setCompliance()`:

```
bytes32 constant EXPECTED_COMPLIANCE_HASH = 0xabcd...;
// ...
function setCompliance(address compliance_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(compliance_.codehash == EXPECTED_COMPLIANCE_HASH, "Unexpected implementation");
    // ...
}
```

1. **[NOVO v2.0] Restrição de pureza:** Garantir que o `AxiaCompliantERC721` trate chamadas para `canTransfer()` como potencialmente perigosas (usando `staticcall` em vez de `call` para garantir que nenhum estado seja alterado):

```
(bool success, bytes memory result) = address(compliance).staticcall(
    abi.encodeCall(compliance.canTransfer, (from, to, amount))
);
require(success, "Compliance check failed");
bool allowed = abi.decode(result, (bool));
```

Correção Aplicada (verificada na v2.0):

A equipe Axia implementou a Recomendação de Mitigação deste finding:

1. A interface `IModularCompliance` passou a expor a função `VERSION()` (`external pure returns (string memory)`), adicionando versionamento explícito.
2. O `AxiaCompliantERC721` passou a validar a versão da implementação em `setCompliance()`, rejeitando implementações com versão não suportada.
3. Foi adicionada a verificação de `codehash` da implementação aprovada em `setCompliance()`, impedindo substituição via SELFDESTRUCT + CREATE2 no mesmo endereço.
4. As chamadas a `canTransfer()` passaram a usar `staticcall`, garantindo que nenhuma alteração de estado ou reentrância ocorra durante a verificação de compliance.
5. A troca de compliance foi colocada sob controle de timelock, eliminando o cenário de upgrade malicioso em transação única.

Verificação da Correção:

- ✓ Confirmado que `VERSION()` está declarada na interface e é validada no `setCompliance()` do consumidor.
- ✓ Confirmado que a verificação de `codehash` e o `staticcall` para `canTransfer()` estão presentes no `AxiaCompliantERC721`.

- ✓ Confirmado que a troca de implementação de compliance está protegida por timelock.

Resposta da Equipe (Opcional): Correção aplicada e confirmada: versionamento, validação de codehash, staticcall e timelock foram implementados conforme recomendado.

[HSEC-MC-02] Funções `created()` e `destroyed()` não distinguem chamador — Risco de Dupla Contagem

Severidade: Medium **Status: Resolved Localização:** `interfaces/IModularCompliance.sol`, Linhas `19`, `21`

Descrição (original): As funções `created(address to, uint256 amount)` e `destroyed(address from, uint256 amount)` não incluem o endereço do contrato chamador (`msg.sender`) como parâmetro. Se múltiplos contratos Axia compartilham a mesma instância de ModularCompliance (arquitetura multi-token T-REX), a implementação pode não conseguir distinguir qual contrato originou a chamada.

Isso pode levar a:

- Contagem incorreta de holders totais (soma entre tokens diferentes).
- Módulos de compliance (ex: max-holders) aplicando limites globais em vez de por-token.
- Impossibilidade de remover um token específico sem afetar os demais.

Reanálise v2.0:

A interface foi atualizada para incluir o parâmetro identificador de token, eliminando a ambiguidade. A análise adicional a seguir fundamentou a correção aplicada:

1. **Cenário multi-token Axia:** Confirmamos que a arquitetura prevê múltiplos tokens (ERC-20 e ERC-721). Caso compartilhem a mesma ModularCompliance sem identificação por token, o módulo `MaxHoldersModule` somaria holders de tokens distintos, causando DoS prematuro.
2. **Uso de msg.sender como workaround:** A implementação concreta pode usar `msg.sender` como identificador. Contudo, **não há garantia na interface** de que a implementação fará isso — e a interface não documenta essa expectativa. Em uma arquitetura onde diferentes implementações de compliance podem ser plugadas, cada uma pode tratar `msg.sender` de forma diferente.
3. **Cenário de Bridge/Relayer:** Se uma bridge ou relayer contratual intermédia chamar `transferred()` em nome do token, `msg.sender` será o contrato de bridge, não o token. Nesse caso, qualquer lógica baseada em `msg.sender` falha.

Recomendação de Mitigação (refinada v2.0):

Opção A (recomendada) — Adicionar `token` como parâmetro explícito:

```
interface IModularCompliance {
    function canTransfer(address token, address from, address to, uint256 amount) external view
returns (bool);
    function transferred(address token, address from, address to, uint256 amount) external;
    function created(address token, address to, uint256 amount) external;
    function destroyed(address token, address from, uint256 amount) external;
}
```

Opção B (menos invasiva) — Documentar expectativa no NatSpec:

Adicionar explicitamente que a implementação DEVE usar `msg.sender` como identificador do token, e o `AxiaCompliantERC721` DEVE chamar diretamente (sem proxies intermediários). Documentar esta restrição como pré-condição no NatSpec.

```
/**
 * @dev IMPLEMENTADORES: msg.sender é o contrato de token chamador.
 *      NÃO DELEGAR a chamada via proxy ou relayer.
 *      CONSUMIDORES: chamar diretamente, sem contratos intermediários.
 */
```

Correção Aplicada (verificada na v2.0):

A equipe Axia implementou a Opção A (recomendada) da Recomendação de Mitigação:

1. As funções `canTransfer`, `transferred`, `created` e `destroyed` passaram a receber o endereço `token` como parâmetro explícito, permitindo à implementação distinguir com precisão qual contrato originou a chamada.
2. Com o identificador explícito, módulos como `MaxHoldersModule` deixam de somar holders de tokens distintos, eliminando a dupla contagem e o risco de DoS prematuro em arquiteturas multi-token.
3. A dependência implícita de `msg.sender` foi removida, tornando a interface robusta também em cenários de bridge/relayer.

Verificação da Correção:

- ✓ Confirmado que as quatro funções da interface agora incluem o parâmetro `token`.
- ✓ Confirmado que o `AxiaCompliantERC721` passa seu próprio endereço como `token` nas chamadas de compliance.
- ✓ Confirmado que a contagem de holders é agora segregada por token, sem soma indevida entre tokens distintos.

Resposta da Equipe (Opcional): Correção aplicada e confirmada: o parâmetro `token` foi adicionado às quatro funções, segregando a contabilização por token.

[HSEC-MC-03] Documentação NatSpec Incompleta — Comportamento de `canTransfer` vs `transferred` não especificado

Severidade: Informational **Status: Unresolved Localização:** `interfaces/IModularCompliance.sol`, Linhas `10-13`

Descrição (original): O comentário na linha 12 afirma que `canTransfer` é `view` e que `transferred/created/destroyed` "DEVEM ser chamadas só após o movimento efetivado". Esta é uma convenção crítica de segurança, mas está apenas em comentário, não no NatSpec formal (`@dev`). Além disso, a interface não especifica:

- Se `canTransfer` e `transferred` devem ser atômicos (i.e., o resultado de `canTransfer` é garantido ser válido até `transferred` ser chamado).
- Se `transferred` pode reverter (e o que acontece com o estado do token nesse caso).
- Se as funções mutantes (`transferred`, `created`, `destroyed`) precisam de controle de acesso.

Reanálise v2.0:

A documentação NatSpec ainda não foi completada; a correção deste item permanece pendente. A análise adicional a seguir fundamenta a recomendação em aberto:

1. **Impacto de falta de atomicidade:** Na ausência de especificação formal, implementadores podem assumir diferentes garantias. Por exemplo, um módulo de compliance pode reverter em `transferred()` após o token já ter efetuado a transferência, deixando o estado do token inconsistente (token transferido mas compliance não atualizada). O token não tem como reverter a transferência nesse ponto.
2. **Documentação para ferramentas formais:** A ausência de NatSpec completo impede que ferramentas de verificação formal (como Scribble ou Certora) extraiam pré/pós-condições automaticamente.

Recomendação de Mitigação (refinada v2.0):

Adicionar documentação NatSpec completa:

```
/**
 * @title IModularCompliance
 * @notice Subset da Modular Compliance do T-REX. Para NFTs, `amount` é sempre 1.
 *
 * @dev IMPLEMENTADORES DEVEM:
 *   - Usar `msg.sender` para identificar o token chamador.
 *   - Garantir atomicidade entre canTransfer e transferred (sem alterações de estado
 *     entre as duas chamadas).
 *   - Restringir `transferred`, `created`, `destroyed` ao contrato de token (msg.sender).
 *   - `canTransfer` é view e NÃO DEVE alterar estado.
```

```
 *   - `transferred` NÃO DEVE reverter após a transferência efetivada no token.
 *     Se houver erro interno, usar eventos de erro em vez de revert.
 *
 * CONSUMIDORES (AxiaCompliantERC721) DEVEM:
 *   - Chamar `canTransfer` ANTES da transferência.
 *   - Chamar `created`/`transferred`/`destroyed` APÓS a transferência efetivada.
 *   - Tratar revert de `transferred` como falha crítica (estado inconsistente).
 *   - Usar `staticcall` para `canTransfer` quando possível.
 */
interface IModularCompliance { ... }
```

Resposta da Equipe (Opcional): Pendente — correção ainda não aplicada pela equipe.

4. Plano de Ação Priorizado (Novo na v2.0)

Com base na reanálise, recomendamos o seguinte plano de ação com prazos sugeridos:

Prioridade	Ação	Finding Relacionado	Esforço Estimado	Prazo Sugerido
🔴 **Crítica**	Adicionar versionamento à interface (`VERSION()`)	HSEC-MC-01	1 hora	Imediato
🔴 **Crítica**	No `AxiaCompliantERC721`, usar `staticcall` para `canTransfer()`	HSEC-MC-01	2 horas	Imediato
🟡 **Alta**	Adicionar parâmetro `token` nas funções `created`/`destroyed`/`transferred`/`canTransfer`	HSEC-MC-02	2 horas	Curto prazo
🟡 **Alta**	Documentar NatSpec completo na interface	HSEC-MC-03	1 hora	Curto prazo
🟢 **Média**	Implementar verificação de codehash em `setCompliance()`	HSEC-MC-01	2 horas	**Médio prazo**
🟢 **Média**	Adicionar testes com implementação maliciosa simulada	HSEC-MC-01, MC-02	8 horas	**Médio prazo**

5. Recomendações Gerais e Melhores Práticas (Atualizado v2.0)

1. **Testes de Integração com Implementação Maliciosa:** Implementar uma suíte de testes Foundry que simule uma ModularCompliance maliciosa (retornando `true`/`false` inesperadamente, revertendo em `transferred`, alterando estado em `canTransfer`) para validar que o `AxiaCompliantERC721` se comporta corretamente em todos os cenários.
2. **Registro de Implementações:** Manter um registro on-chain das implementações de compliance aprovadas, similar ao padrão ERC-1820 (Registry), para que terceiros possam verificar a legitimidade.
3. **Eventos de Auditoria:** A implementação concreta deve emitir eventos em `created`, `destroyed` e `transferred` para permitir rastreamento off-chain.
4. **[NOVO v2.0] Staticcall para canTransfer:** Conforme detalhado em HSEC-MC-01, recomenda-se fortemente o uso de `staticcall` para garantir que `canTransfer()` não altere estado, independentemente da implementação concreta.
5. **[NOVO v2.0] Fuzz Testing de Assinaturas:** Implementar testes fuzz que verifiquem se as assinaturas da interface são compatíveis com a implementação concreta esperada (e.g., testar se `abi.encodeCall` funciona corretamente para todas as funções).
6. **Compatibilidade T-REX:** Validar que a implementação concreta utilizada (Tokeny ou custom) segue estritamente a especificação ERC-3643 para ModularCompliance, incluindo os módulos padrão (CountryAllowModule, MaxHolderModule, etc.).

6. Conclusão da Revisão 2.0

Após a segunda revisão completa da interface `IModularCompliance.sol`, concluímos que **a equipe Axia aplicou as correções recomendadas para os findings Medium** da auditoria inicial (v1.0, 18/06/2026). Consequentemente:

- **Os 2 findings Medium foram resolvidos e verificados** — 2 de 3. O finding Informational / Gas (HSEC-MC-03) **permanece em aberto**, com correção ainda não aplicada pela equipe.
- **Nenhum novo vulnerability foi descoberto** — a análise confirmou a eficácia das mitigações aplicadas aos findings Medium.
- **As recomendações dos findings Medium foram integralmente atendidas**, incluindo versionamento, verificação de codehash, staticcall e parâmetro `token`. A documentação NatSpec formal recomendada em HSEC-MC-03 ainda está pendente.

Risco Residual: Baixo a moderado. Com as correções Medium aplicadas, a interface e seu contrato de integração passaram a contar com versionamento, verificação de codehash, chamadas via `staticcall` e identificação explícita de token. As proteções de transferência estão adequadamente reforçadas contra os cenários de implementação maliciosa e dupla contagem originalmente identificados. Permanece pendente a documentação NatSpec formal (HSEC-MC-03), item de natureza informativa cuja ausência não introduz vulnerabilidade direta, mas cuja resolução é recomendada para completar as garantias de segurança e permitir a verificação formal.

7. Termo de Isenção de Responsabilidade (Disclaimer)

Este relatório de auditoria não constitui um endosso, garantia ou certificação absoluta de que o código auditado está livre de vulnerabilidades ou falhas. A auditoria representa um esforço de tempo limitado (time-boxed) para identificar o máximo possível de problemas de segurança utilizando o conhecimento e as ferramentas disponíveis no momento da execução.

A HeimdallSec não se responsabiliza por perdas financeiras, danos operacionais ou explorações futuras que possam ocorrer no protocolo.

gov.br
Documento assinado digitalmente
KLEBER AQUINO DOS SANTOS JUNIOR
Data: 24/07/2026 21:17:37-0300
Verifique em https://validar.iti.gov.br